CS3021 Introduction to Data Structures and Intermediate Programming

Searching Sequences

Searching

- When we maintain a collection of data, one of the operations we need is a search routine to locate desired data quickly.
- We will use an array of integers to present searching algorithms.
- Here is the problem statement (we assume no duplicate entries in the array):

  Given a value X, return the index of X in the array, if X exists.
  Otherwise, return NOT_FOUND (-1).

Search Result

Unsuccessful Search: `search(45)` → ***NOT_FOUND***

Successful Search: `search(12)` → **4**

number

0	1	2	3	4	5	6	7	8
23	**17**	**5**	**90**	**12**	**44**	**38**	**84**	**77**

Linear Search

- Search the array from the first to the last position in linear order

```
int linearSearch (int number[], int length, int searchValue)
{
    int loc = 0;
    while (loc < length && number[loc] != searchValue) {
        loc++;
    }
    if (loc == length) {  // searchValue not found
        return NOT_FOUND;
    }
    else {
        return loc;   // searchValue found, return its position
    }
} // end linearSearch
```

Linear Search Performance

- We analyze the successful and unsuccessful searches separately by counting how many times the search value is compared against the array elements (N = array size).
- Successful Search:
  - Best Case = 1 comparison
  - Worst Case = N comparisons
  - Average Case = $N/2$ comparisons
- Unsuccessful Search:
  - Best Case = Worst Case = N comparisons

Binary Search

- If the array is sorted, then we can apply the *binary search* technique.

number

0	1	2	3	4	5	6	7	8
5	12	17	23	38	44	77	84	90

- First, check the value in the middle position. If X is less than this value, then search the middle of the left (lower) half next. If X is greater than this value, then search the middle of the right (upper) half next. Continue until value found (or not).

Sequence of Successful Search (1)

	low	high	mid
#1	0	8	4

search(44)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

0	1	2	3	4	5	6	7	8
5	12	17	23	38	44	77	84	90
low				mid				high

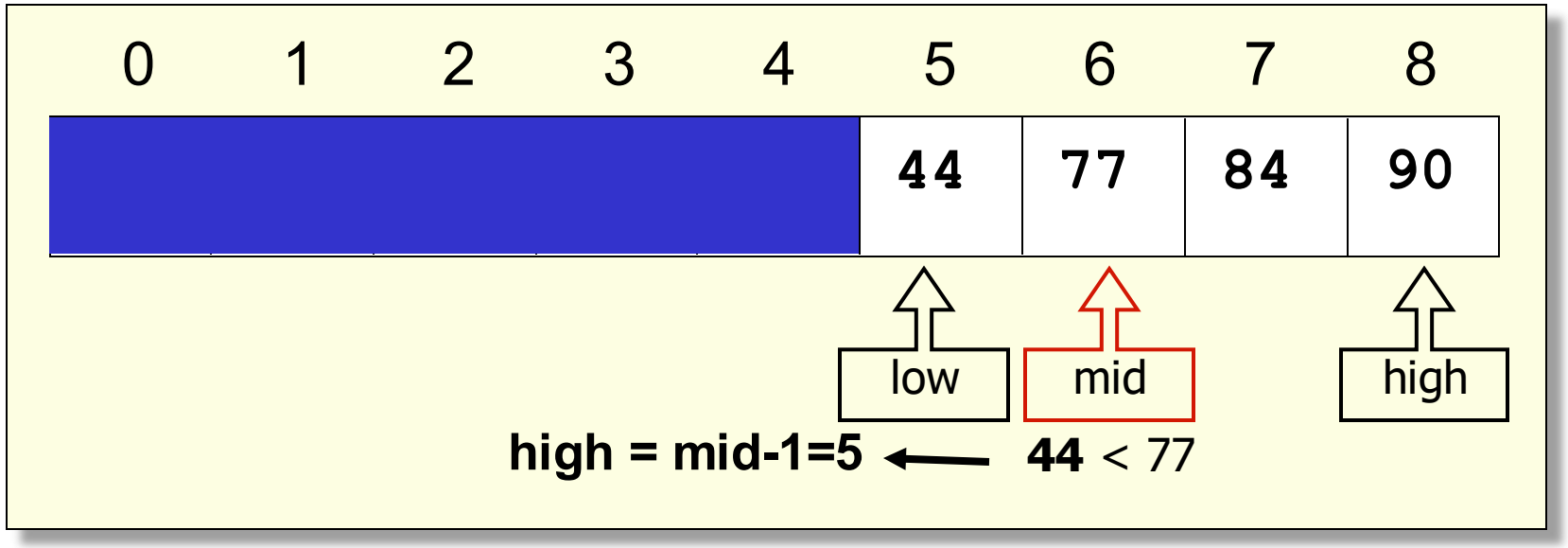
38 < **44** → **low = mid+1 = 5**

Sequence of Successful Search (2)

	low	high	mid
#1	0	8	4
#2	5	8	6

search(44)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

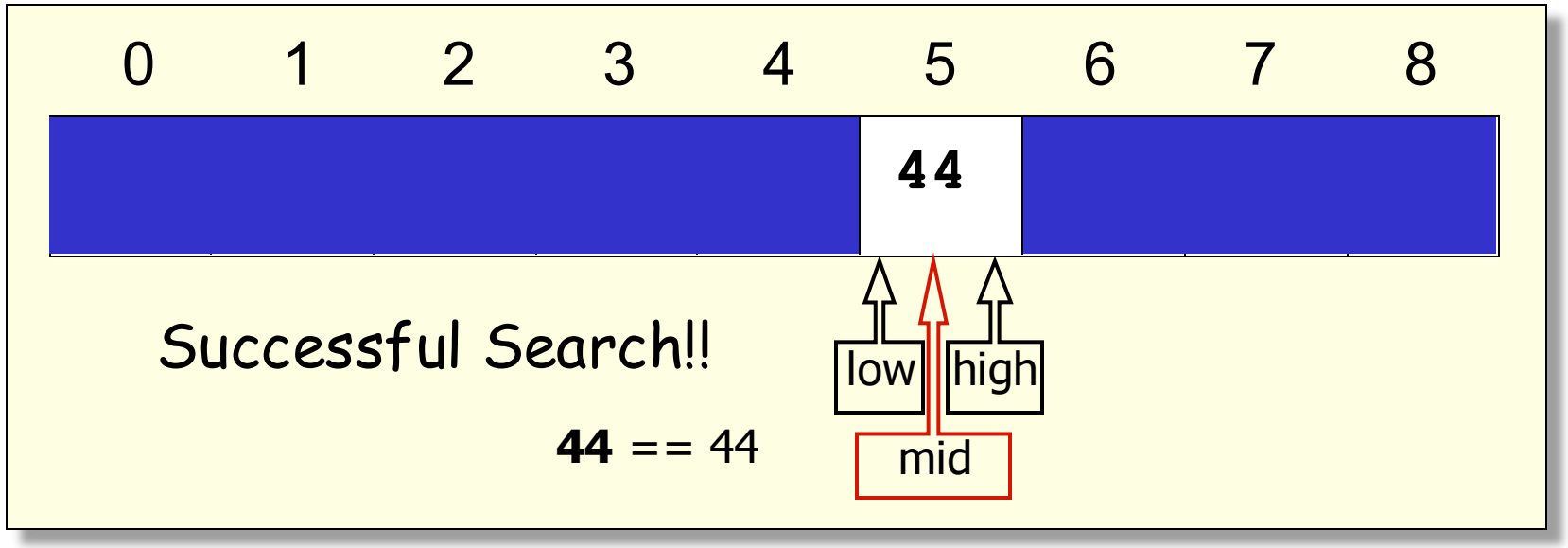


Sequence of Successful Search (3)

	low	high	mid
#1	0	8	4
#2	5	8	6
#3	5	5	5

search(44)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$



Sequence of Unsuccessful Search (1)

	low	high	mid
#1	0	8	4

search(45)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

0	1	2	3	4	5	6	7	8
5	12	17	23	38	44	77	84	90
low				mid				high

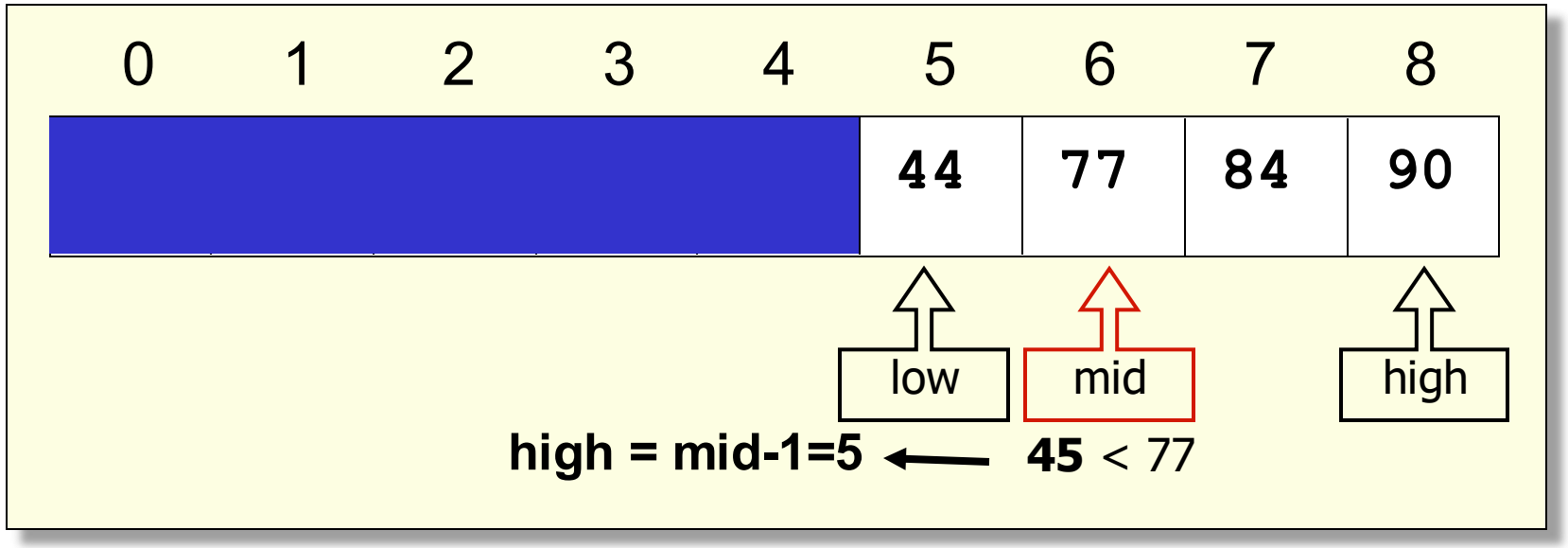
38 < **45** → **low = mid+1 = 5**

Sequence of Unsuccessful Search (2)

	low	high	mid
#1	0	8	4
#2	5	8	6

search(45)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

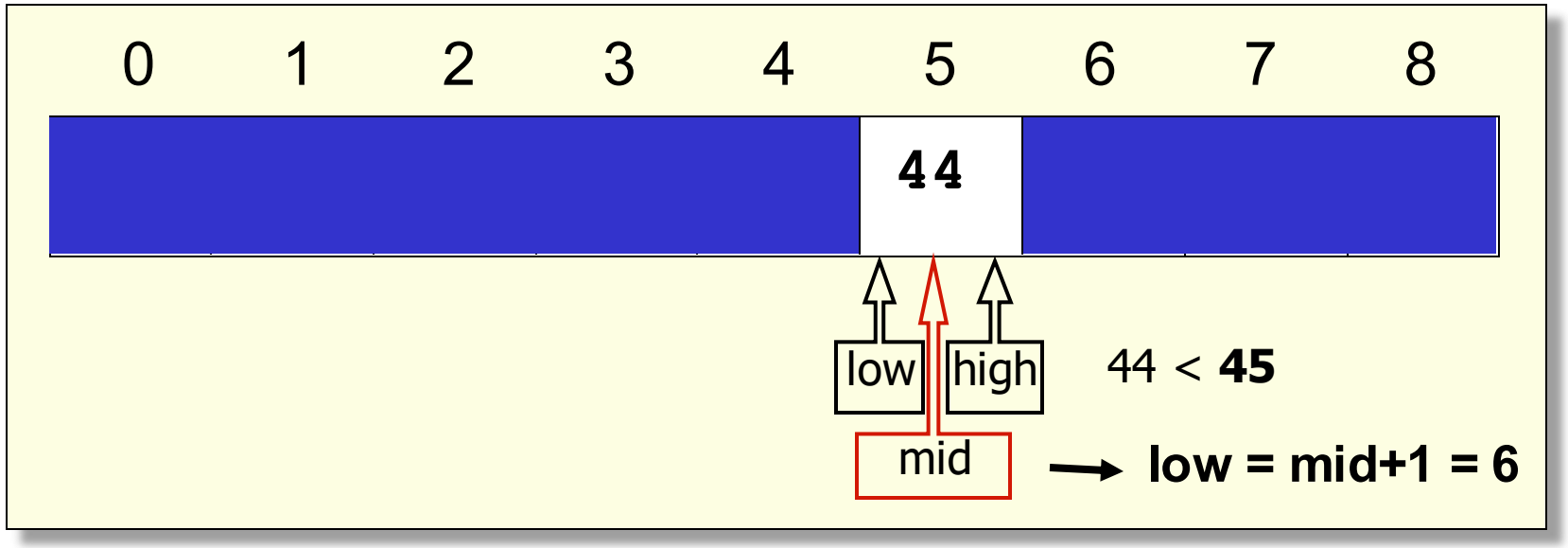


Sequence of Unsuccessful Search (3)

	low	high	mid
#1	0	8	4
#2	5	8	6
#3	5	5	5

search(45)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

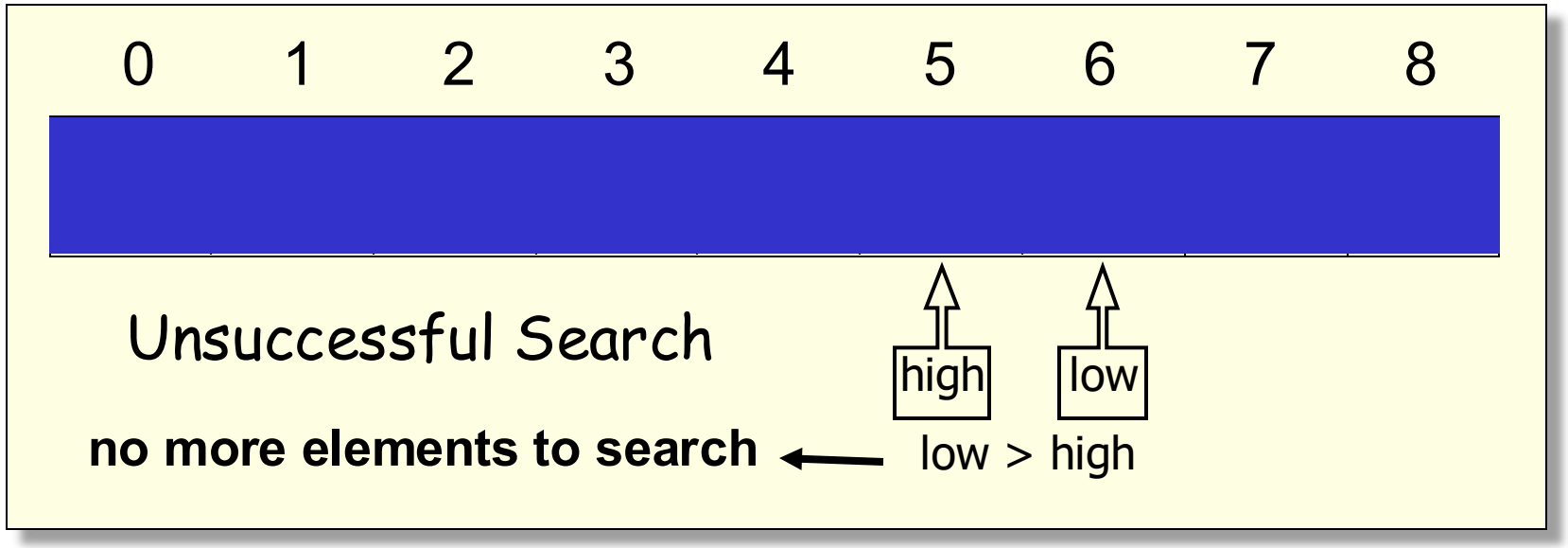


Sequence of Unsuccessful Search (4)

search(45)

	low	high	mid
#1	0	8	4
#2	5	8	6
#3	5	5	5
#4	6	5	

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$



Binary Search Routine

```
int binarySearch (int number[], int length, int searchValue)
{
    int low  = 0,
        high = length - 1,
        mid  = (low + high) / 2;  // integer div will truncate

    while (low <= high && number[mid] != searchValue) {
        if (number[mid] < searchValue) {
            low = mid + 1;
        }
        else  {   // number[mid] > searchValue
            high = mid - 1;
        }
        mid = (low + high) / 2;
    }

    if (low > high) {
        mid = NOT_FOUND;
    }
    return mid;  // searchValue found at mid
} // end binarySearch
```

Binary Search Performance

- Since the portion of an array to search is cut into half after every comparison, we compute how many times the array can be divided into halves.
- After K comparisons, there will be $N/2^K$ elements in the list. We solve for K when $N/2^K = 1$, deriving $K = \log_2 N$
- Successful Search:
  - Best Case = 1 comparison
  - Worst Case = $\log_2 N$ comparisons
- Unsuccessful Search:
  - Best Case = Worst Case = $\log_2 N$ comparisons

Questions?